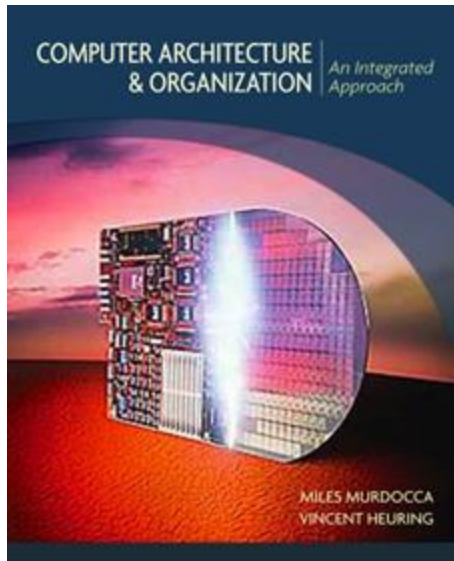


Organización de las Computadoras

Carolina Chaves Garcia



Programación directa (2)

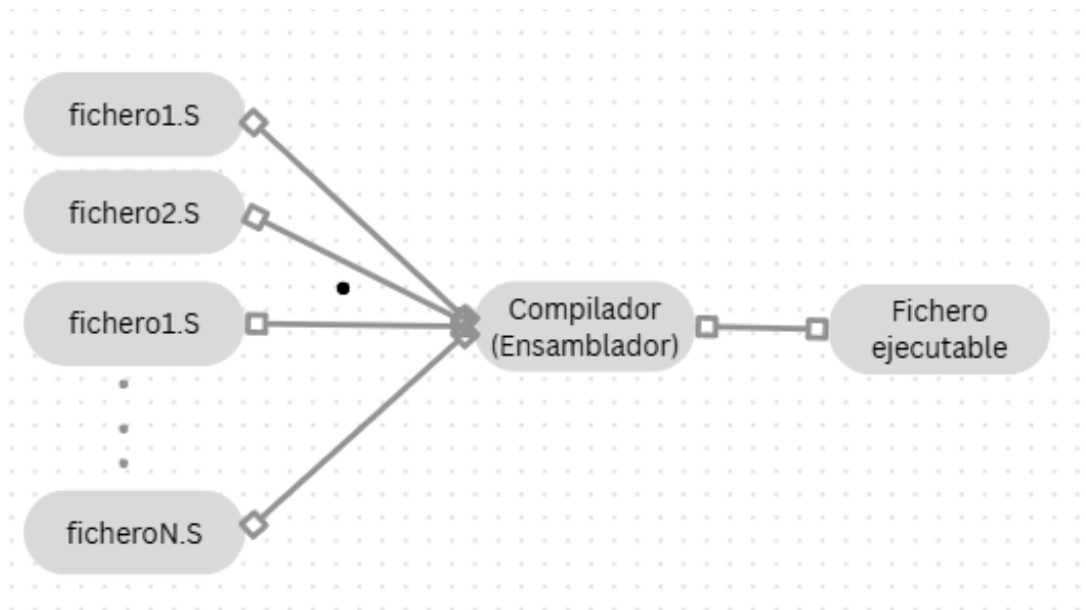
Niveles de lenguaje

Niveles de lenguaje

- **Hay 4 niveles fundamentales:**
 - Lenguaje de Alto Nivel (C, Python, Java)
 - Abstracto, cercano al ser humano
 - $res = a + b * c$
 - Lenguaje Ensamblador (RISC-V ASM)
 - Mnemotécnicos comprensibles
 - `add t0, a0, a`
 - Lenguaje de Máquina
 - Instrucciones en código máquina
 - `0x00A58533`
 - Microarquitectura
 - Circuitos que ejecutan instrucciones

Niveles de lenguaje

- Los programas escritos en lenguaje ensamblador -> representan instrucciones del lenguaje máquina del procesador. Sin embargo no son ejecutables directamente por el computador, se deben traducir a binario.
- Este proceso se realiza por programas denominados ensambladores (compiladores).
- El ensamblador recibe como datos de entrada los diferentes programas de texto plano (ficheros) con el conjunto de instrucciones y datos escritos en lenguaje assembler, y genera un fichero binario y uno ejecutable.



Niveles de lenguaje

Assembler

- Las palabras que comienzan con punto (`.data`, `.text`, etc) se les conoce como directivas.
- La línea siguiente contiene la directiva `.text` que denota el comienzo de la sección de código. La directiva `.global main` comunica al ensamblador que la etiqueta `main` es globalmente accesible desde cualquier otro programa. indica el punto de arranque del programa.

Todo programa ensamblador debe seguir el siguiente patrón:

```
.data                # Comienzo del segmento de datos
<datos del programa>
```

```
.text                # Comienzo del código
.global main         # Obligatorio
```

```
main:
<Instrucciones>
ret                  # Obligatorio
```

Niveles de lenguaje

Assembler

Definición de bytes: `.byte` valores numéricos almacenados como bytes

datos: `.byte 38, 0b11011101, 0xFF, 'A', 'b'`

Definición de enteros: `.int`

nums: `.int 3, 4, 5`

`.int 0x12AB, 0x10ab, 0b111000, 0B111000`

`.int 21`

`.int 07772`

La directiva `.long` es un sinónimo de `.int` y también define enteros de 32 bits. Las directivas `.word` 32bits .

Niveles de lenguaje

Assembler

Definición de strings: Se puede realizar de dos formas

- **.ascii**
 - que nos permite definir uno o más strings entre comillas y separándolos por comas.
 - Cada símbolo de cada strings es codificado con un byte en ASCII utilizando posiciones consecutivas de memoria.
 - Se utilizan tantos bytes como la suma de los símbolos de cada string.
- **.asciz**
 - tambien permite escribir uno o más strings separados por comas, pero cada uno de ellos se codifica añadiendo un byte con valor cero a final del string.
 - Este formato se suele utilizar para detectar el final del string.
- La directiva **.string** es un sinónimo de la directiva **.asciz**.

Niveles de lenguaje

Assembler

Definición de espacio en blanco

- La directiva `.space` permite reservar espacio en memoria, debe estar seguida de dos números separados por una coma.
- El primer valor indica el número de bytes que deseamos reservar y el segundo valor inicializa esos bytes (0 -255). Si se omite ese valor la memoria se reserva, pero no se inicializa en ningún valor
- Se usa principalmente para reservar espacio que se debe inicializar al mismo valor, o que su valor será calculado y modificado por nuestro programa.

result: `.space 4, 0`

`.space 4`

`.space 8, 0xFF`

- Uso de etiquetas: Son un punto de referencia en memoria que el ensamblador sabe interpretar y que en la ejecución será reemplazado por el valor numérico de la dirección de memoria correspondiente.

Organización de la memoria

Organización de la memoria

0x00000000		
	Text	Código del programa
	Data	Variables globales inicializadas
	BSS	Variables globales no inicializadas
	Heap	Memoria dinámica (malloc) ↑ (crece hacia direcciones altas)
	Stack	Variables locales ↓ (crece hacia direcciones bajas)
0x7FFFFFFF		

Organización de la memoria

Registros Críticos RISC-V:

- sp (x2): Stack Pointer → Apunta al tope del stack
- ra (x1): Return Address → Guarda dirección de retorno
- a0-a7 (x10-x17): Argumentos y retornos (a0 y a1: Guardan los dos primeros argumentos de la función o los valores devueltos - a2-a7: Guarda el resto de los argumentos)
- s0-s11: Valores que persisten después de las llamadas a funciones
- t0-t6 (x5-x7, x28-x31): Registros temporales → Guarda valores que no persisten después de las llamadas a funciones

Uso del stack:

- Se utiliza como depósito temporal de datos del programa en ejecución.
- ¿Por qué el stack crece hacia abajo?
 - Permite que heap y stack crezcan sin colisionar
 - Direcciones altas para heap, bajas para stack

Organización de la memoria

- **Push** y **Pop** permiten depositar y obtener datos de la pila, pero no son la únicas que modifican su contenido

.data

valor1: .word 10

valor2: .word 20

.text

.global main

main:

Push de valores al stack

lw t0, valor1 # Cargar valor1 en t0

addi sp, sp, -4 # Reservar espacio en stack

sw t0, 0(sp) # Guardar t0 en stack

lw t1, valor2 # Cargar valor2 en t1

addi sp, sp, -4 # Reservar espacio

sw t1, 0(sp) # Guardar t1 en stack

Organización de la memoria

Pop de valores del stack

lw t2, 0(sp) # Cargar último valor

addi sp, sp, 4 # Liberar espacio

lw t3, 0(sp) # Cargar primer valor

addi sp, sp, 4 # Liberar espacio

Organización de la memoria

Llamadas a funciones

- Principalmente se realizan con la instrucción jal (Jum And Link) o jalr (Jump And Link Register).
- Instrucciones de Llamada
 - jal rd, etiqueta
 - Salta a la etiqueta de la función y guarda la dirección de la siguiente instrucción (después del jal) en el registro rd, que usualmente es ra.
 - jalr rd, etiqueta:
 - Este, similar a jal, pero salta a la dirección especificada por etiqueta o registro (jalr rd, rs), donde el salto se realiza a la dirección contenida en el registro rs.
 - ret se usa para saltar a la dirección guardada en ra
 - Si hay una llamada anidada (función que llama a otra función) ra se sobrescribe.
- Pasaje de Argumentos
 - Los registros a0 a a7 se usan para los argumentos de una función. (a0-a7: Primeros 8 argumentos)
 - Cuando tenemos más de 8 argumento, o en caso que tengan tamaño grande, se pueden pasar usando la pila. Stack: Argumentos adicionales.
 - a0-a1: Valores de retorno

Organización de la memoria

Llamadas a funciones

.text

.globl main

Función (subrutina) llamada: $a0 + a1 \rightarrow a0$

suma:

add a0, a0, a1 # $a0 = a0 + a1$ Operación principal

ret # retorno a dirección en ra - retorna al caller

main:

Preparar parámetros

li a0, 5 # primer argumento = 5

li a1, 3 # segundo argumento = 3

Llamar subrutina

call suma # call guarda dirección de retorno en ra

Organización de la memoria

Carga y almacenamiento de datos de memoria

- Las instrucciones de carga y almacenamiento acceden a la memoria.
- Las de carga leen un valor de la memoria y lo guardan en un registro.
- Las de almacenamiento escriben el valor de un registro en la memoria.
- Tamaño de datos:
- Estas instrucciones admiten datos de 8 y 16 bits

.data

array: .word 10, 20, 30, 40, 50

resultado: .word 0

.text

.globl main

main:

Carga (Load)

la t0, array # Cargar dirección de array en t0

lw t1, 0(t0) # Cargar primer elemento (10)

lw t2, 4(t0) # Cargar segundo elemento (20)

Almacenamiento(Store)

add t3, t1, t2 # Sumar valores

la t4, resultado # Cargar dirección de resultado

sw t3, 0(t4) # Almacenar resultado en memoria

Gracias